How Python Executes a Function

1. Modularisation and functions

What it means

Modularisation means splitting a program into small, self-contained units, each with one clear job, and building the whole program by combining them. Instead of one long script that does everything, you write many small pieces that each do one thing well and are easy to name, test and reuse.

How functions help

- A function gives a block of code a **name**, so you call it instead of copying it.
- Parameters are the **inputs** and the return value is the **output**. Callers need to know only this contract, not the internals.
- Each call runs in its own frame, so its local variables cannot clash with anyone else's. This is what makes small units safe to combine (explained in sections 4 and 5).

Without functions

The same tax calculation is copied for every item:

```
# Without functions: the same logic is copied for every item
price1, qty1 = 250, 2
price2, qty2 = 99, 5
print("Item 1:", round(price1 * qty1 * 1.18, 2))
print("Item 2:", round(price2 * qty2 * 1.18, 2))
```

```
OUTPUT
Item 1: 590.0
Item 2: 584.1
```

It works, but the formula `* 1.18` is now written in two places. With 50 items it would live in 50 places, and a rate change or a bug fix means editing every copy.

With functions

Each job gets its own function, and `bill()` simply combines them:

```
GST = 0.18

def line_total(price, qty):
    return price * qty

def with_tax(amount):
    return round(amount * (1 + GST), 2)

def bill(items):
    total = 0
    for price, qty in items:
        total += with_tax(line_total(price, qty))
    return round(total, 2)

print("Bill:", bill([(250, 2), (99, 5)]))
```

```
OUTPUT
Bill: 1174.1
```

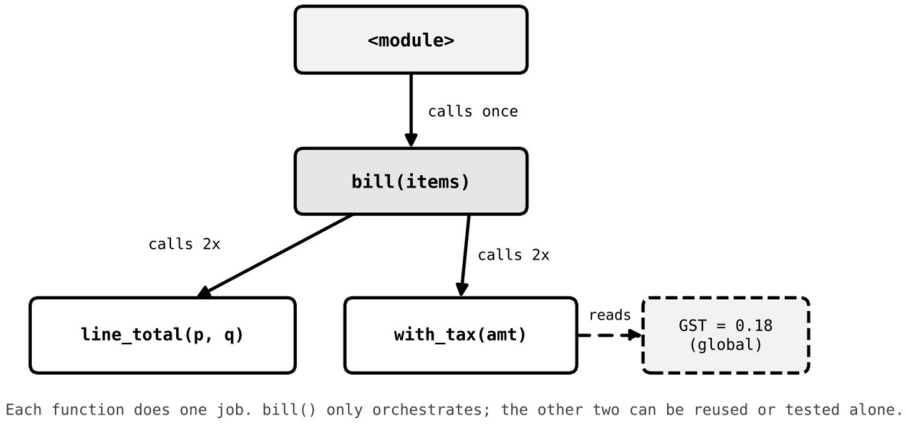


Figure 1. Call structure of the modular program.

How it executes

`bill([(250, 2), (99, 5)])` runs like this. Every row that says "push" creates a new frame, and every "pop" discards one:

#	What happens	Stack after the step (bottom to top)
1	Push `bill`'s frame. Its locals are `items=[(250, 2), (99, 5)]` and `total=0`.	`<module>, bill`
2	Loop takes (250, 2): price=250, qty=2 (locals of bill).	`<module>, bill`
3	Call `line_total(250, 2)`: push a frame with its own `price=250`, `qty=2`. It runs `return price * qty`, returns 500, and its frame pops.	`<module>, bill, line_total`
4	Call `with_tax(500)`: push a frame with `amount=500`. `GST` is not local, so it is read from the globals. It returns 590.0, and the frame pops.	`<module>, bill, with_tax`
5	total += 590.0, so total = 590.0.	`<module>, bill`
6	The loop repeats for (99, 5): `line_total` gives 495 and `with_tax` gives 584.1, so `total` = 590.0 + 584.1 = 1174.1.	`<module>, bill`
7	Return round(total, 2) = 1174.1. bill's frame pops, and the module prints it.	`<module>`

Note that `price` and `qty` exist both in `bill` (loop variables) and in `line_total` (parameters). They are separate variables in separate frames, so neither disturbs the other.

Benefits

Benefit	Why
Reuse (DRY)	Write the logic once, call it from anywhere.
Readability	bill() reads like a sentence. Details are hidden behind names.
Easy maintenance	Change GST or fix a bug in one place and every caller benefits.
Testability	line_total(250, 2) == 500 can be checked without running the rest.
Abstraction	Callers depend on what a function does, not how it does it.
Safe local scope	Locals live only in their own frame, so names do not collide.
Teamwork	Different people can build and change different functions independently.

Benefit	Why
Easier debugging	A traceback names the exact chain of functions that led to the error.

2. The function object (created at def time)

```
x = 10
def add(a, b):
    c = a + b
    return c + x
```

`def` does not run the body. It builds a **function object** and binds the name `add` to it in the enclosing namespace. That object holds:

- `__code__`: the compiled code object (bytecode, immutable, shared by every call)
- `__globals__`: a reference to the module's global dict
- `__defaults__`, `__closure__`: default arguments and captured variables

The function object is the **blueprint**. Nothing about a particular call lives in it. The body runs only when the function is called (next section).

3. A call creates a frame

`add(2, 3)` creates a **frame**, the runtime workspace for that one call. A frame holds:

Part	Purpose
`fast locals array`	Slots for a, b, c, read by index with LOAD_FAST
`value stack`	Scratch space used while evaluating bytecode
`f_code`	The code object being run
`f_globals / f_builtins`	Where non-local names are looked up
`f_back`	Link to the caller's frame
`f_lasti`	Index of the current bytecode instruction

Putting it together, the call `add(2, 3)` goes through these steps:

- **Look up**: Python evaluates the arguments and finds the function object bound to `add`.
- **Create the frame**: a new frame is built and `a=2`, `b=3` are stored in its locals.
- **Run the body**: the frame is pushed and the bytecode runs. `c = a + b` gives 5. In `return c + x`, `x` is not local, so it is found in the globals (10), giving 15.
- **Return**: the frame is popped, its locals are released, and the caller resumes with 15.

Because `a`, `b`, `c` live only in this frame's slots, no other frame can see them. When the function returns, the frame is discarded and its locals are released.

Name lookup follows **LEGB**: **Local** (frame) → **Enclosing** (closure) → **Global** (`f_globals`) → **Built-in**.

4. House and rooms

We can think of frames as rooms in a house. The module is a **house** and each function call is a **room**. The base of every call stack is the “Global frame”, which we can picture as the hall: every room opens onto it, so any function can look into the hall and read the module’s variables. When a function is called from the hall, a new room is created for that call, and when the function finishes, the room is removed. A function called from inside another function is a room inside a room.

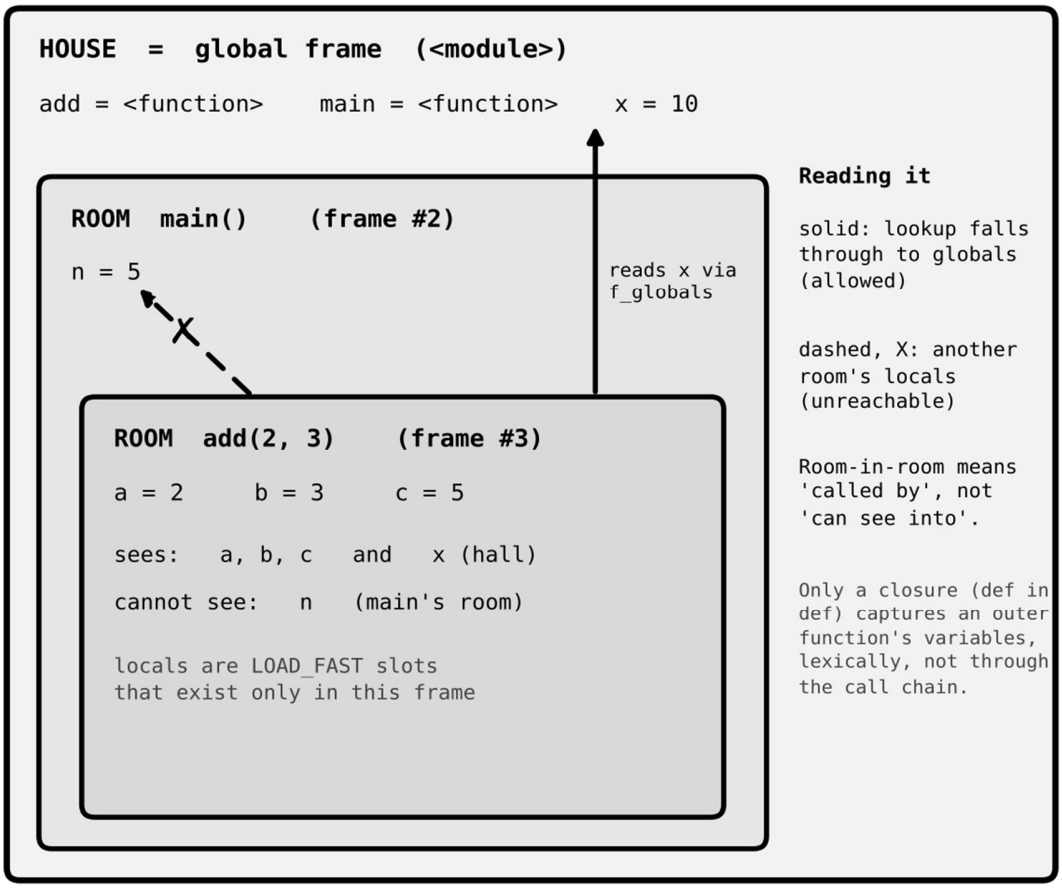


Figure 2. Global frame as a house, call frames as rooms. Solid arrow = allowed lookup, dashed arrow with X = unreachable.

If the global and local variables have different names, you can **read** the global variable directly by its name without any special keywords, though you still need the `global` keyword if you want to **reassign** it.

However if a function has a variable with the same name as a global variable, then you can access the global variable using the built-in `globals()` function if you need to keep both variables separate, or use the `global` keyword if you want the function to use the global variable directly instead of creating a local one.

1. Using `globals()` (When you need both variables at the same time) The `globals()` function returns a dictionary of the current module's global variables. Accessing the variable by its string name lets you read—and even modify—the global variable while keeping a separate local variable of the same name.

```
count = 10  # Global variable

def update():
    count = 5  # Local variable (shadows the global one)

    print("Local count:", count)              # Output: 5
    print("Global count:", globals()['count']) # Output: 10

    # You can also modify the global variable without changing the local one
    globals()['count'] = 20

update()
print("Global count after function:", count)   # Output: 20
```

2. Using the `global` keyword (When you don't actually want a separate local variable) If your goal is to read and reassign the global variable inside the function rather than maintaining two separate variables, declare `global <variable_name>` at the start of the function. This tells Python not to create a local variable at all.

```
count = 10  # Global variable

def update():
    global count  # Refers directly to the global variable
    count = 5     # Modifies the global variable directly
    print("Inside function:", count) # Output: 5

update()
print("Outside function:", count)    # Output: 5
```

---

5. What actually happens: the frame stack

The nested rooms are really a **stack** of frames linked by `f_back`. The newest call is on top and only the top frame runs. All frames share one globals dict, and every call to the same function shares one function object and one code object.

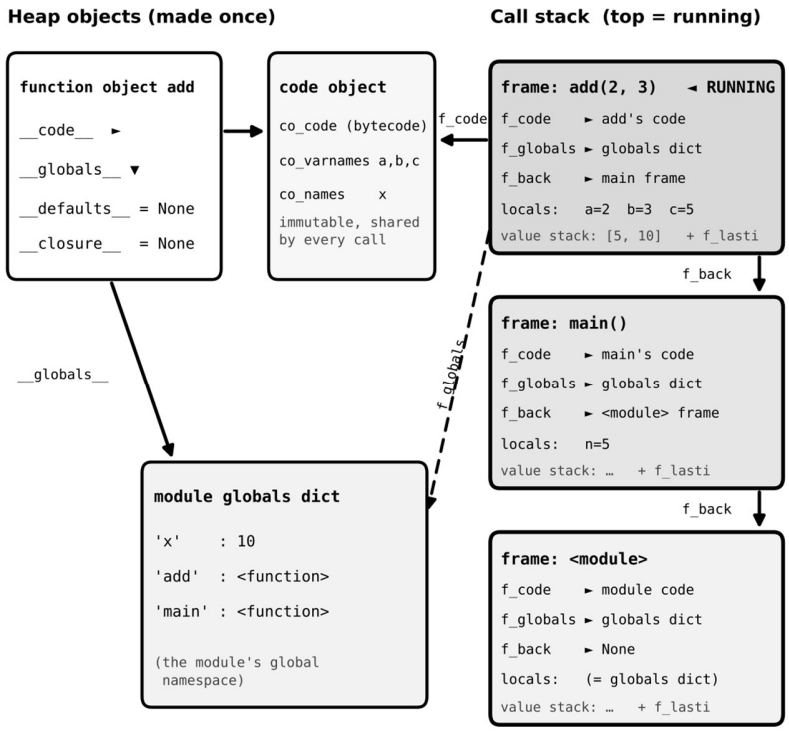


Figure 3. Heap objects (left) and the call stack (right) while add(2, 3) is running.

- **Call**: arguments are bound to the new frame's locals, the frame is **pushed** and becomes the running frame.
- **Return**: the frame is **popped**, its locals are released, and the caller resumes at the call with the return value.
- Uncaught exception: frames unwind one by one until a handler is found. A traceback is a walk down the `f_back` chain.

You can watch the stack grow. `show()` uses `inspect.stack()` to print the chain of frames, bottom first:

```
import inspect

def show(label):
    names = [f.function for f in inspect.stack()][::-1]
    print(f"{label:<12}", " > ".join(names))

def total(nums):
    show("in total")
    return sum(nums)

def average(nums):
    show("in average")
    return total(nums) / len(nums)

def main():
    show("in main")
    print("average =", average([2, 4, 6]))

main()
```

OUTPUT

```
in main      <module> > main > show
in average   <module> > main > average > show
in total     <module> > main > average > total > show
average = 4.0
```

Each line is the stack at that instant. `show` itself is the top frame while it runs, and `<module>` is always the bottom. By the time `total` runs there are five frames. As each function returns, its frame disappears and the line gets shorter again.